

Université Côte d'Azur

M1 Informatique Générale

Projet Recherche Opérationnelle

Yunus Emre DOGAN

1 Introduction

Dans ce projet, nous implémentons plusieurs algorithmes de flots sur des graphes : le flot maximum avec Ford-Fulkerson, le flot à coût minimum avec deux variantes (Bellman-Ford et Dijkstra), ainsi qu'une détection de cycles négatifs. Ces problèmes ont de nombreuses applications concrètes : optimisation de réseaux de transport, de télécommunications ou de chaînes logistiques. L'objectif est de proposer une implémentation correcte et efficace de ces algorithmes sur un graphe résiduel.

2 Structure de données

On a implémenté 4 classes principales.

1) **Noeud** représente un sommet du graphe, identifié par un entier.

2) **Arc** représente un arc du graphe. Un arc contient :

- deux sommets `N1` et `N2`
- une capacité maximale `max_capacity`
- un flot courant `flow`, initialisé à 0
- un coût unitaire `cost` (utilisé uniquement pour le flot à coût minimum)
- une référence vers l'arc inverse `reverse_arc` : cet attribut est indispensable dans le graphe résiduel pour pouvoir annuler les décisions prises lors de l'augmentation de flot, sans avoir à parcourir toute la liste d'adjacence.

La capacité résiduelle d'un arc est calculée par :

$$capacite_residuelle = max_capacity - flow$$

3) **MaxFlow** contient les méthodes de l'algorithme Ford-Fulkerson. Le graphe est représenté par une liste d'adjacence de type `ArrayList<Arc>[]` : pour chaque sommet i , `adj_list[i]` contient la liste des arcs sortants. Ce choix évite de créer une matrice d'incidence, ce qui est avantageux pour les graphes creux car la complexité en espace est $O(V + E)$ au lieu de $O(V^2)$.

Lors de l'ajout d'un arc, on crée automatiquement son arc inverse avec une capacité maximale de 0 et un coût de 0. Les deux arcs se référencent mutuellement via `reverse_arc`.

4) **MinCostGraph** hérite de **MaxFlow** et surcharge la méthode `add_arc` pour créer les arcs inverses avec un coût négatif ($-cost$), ce qui est nécessaire pour le flot à coût minimum. Le flot à coût minimum consiste à envoyer un flot de S vers T en minimisant le coût total.

3 Algorithmes

3.1 Ford-Fulkerson

Pour trouver le flot maximum, on utilise l'algorithme Ford-Fulkerson. A chaque itération, on cherche un chemin augmentant de S à T dans le graphe résiduel avec un DFS. Si un tel chemin existe, on calcule le flot maximum qu'on peut envoyer sur ce chemin, qui correspond à la capacité résiduelle minimale des arcs du chemin (le bottleneck). On augmente ensuite le flot sur les arcs du chemin et on diminue le flot sur les arcs inverses. On répète jusqu'à ce qu'il n'existe plus de chemin augmentant.

3.2 Coupe minimale

Après avoir trouvé le flot maximum, on trouve la coupe minimale en faisant un DFS depuis S dans le graphe résiduel. On récupère tous les noeuds atteignables depuis S. La coupe minimale est constituée de tous les arcs allant d'un noeud atteignable vers un noeud non atteignable ayant une capacité maximale strictement positive.

3.3 Flot à coût minimum

Le flot à coût minimum consiste à envoyer un flot de S vers T en minimisant le coût total. On a implémenté deux variantes.

3.3.1 Variante Bellman-Ford

A chaque itération, on utilise Bellman-Ford pour trouver le chemin augmentant de coût minimum. On initialise les distances de tous les noeuds à l'infini sauf S dont la distance est 0. Ensuite on met à jour les distances de tous les arcs $V - 1$ fois : si $d(u) + c_{uv} < d(v)$, on met à jour $d(v) = d(u) + c_{uv}$. Une fois le chemin trouvé, on reconstruit le chemin de T vers S grâce au tableau `parent`, on calcule le flot maximum à envoyer et on met à jour les flots.

3.3.2 Détection de cycles négatifs

Parfois un graphe peut contenir des cycles négatifs. Si c'est le cas, la solution optimale n'existe pas car on peut toujours envoyer plus de flot pour réduire le coût total. On détecte un cycle négatif en faisant une itération supplémentaire de Bellman-Ford après les $V - 1$ itérations. Si une distance est encore mise à jour, cela signifie qu'il existe un cycle négatif dans le graphe résiduel.

3.3.3 Variante Dijkstra avec renormalisation

Dijkstra ne fonctionne pas avec des arcs de coût négatif. Pour contourner ce problème, on utilise une renormalisation des coûts. On commence par calculer les potentiels initiaux d avec Bellman-Ford une seule fois au début car il peut y avoir des arcs négatives dans le graphe. Ensuite à chaque itération, on utilise le coût réduit :

$$c_{(u,v)}^R = c_{(u,v)} + d(u) - d(v)$$

Ce coût réduit est toujours positif ou nul, ce qui permet à Dijkstra de fonctionner correctement. On utilise une file à priorité pour extraire à chaque étape le noeud ouvert

ayant la plus petite distance, ce qui donne une complexité de $O((V + E) \log V)$. Après chaque itération, on met à jour les potentiels d avec les distances trouvées par Dijkstra.

4 Complexité

4.1 Variante Bellman-Ford

Bellman-Ford parcourt tous les arcs E pour chaque sommet V , ce qui donne une complexité de $O(V \cdot E)$ par itération du Successive Shortest Path.

4.2 Variante Dijkstra avec renormalisation

Dijkstra avec une file à priorité a une complexité de $O((V + E) \log V)$ par itération.

5 Comparaison des deux variantes

Les deux variantes produisent le même résultat optimal. La différence est purement en termes de performance.

Critère	Bellman-Ford	Dijkstra
Complexité par itération	$O(V \cdot E)$	$O((V + E) \log V)$
Coûts négatifs	Oui	Non (renormalisation nécessaire)
Graphes creux	Moins efficace	Plus efficace
Implémentation	Plus simple	Plus complexe
Détection de cycles négatifs	Oui	Non

TABLE 1 – Comparaison entre les variantes Bellman-Ford et Dijkstra

En théorie, Dijkstra est plus efficace grâce à sa complexité $O((V + E) \log V)$, nettement inférieure à $O(V \cdot E)$ de Bellman-Ford. Sur de grands graphes, cet écart devient significatif : Bellman-Ford peut devenir lente là où Dijkstra reste performant. En pratique cependant, la présence de coûts négatifs dans le graphe résiduel impose une renormalisation préalable, ce qui introduit un surcoût supplémentaire. Le choix final dépend donc du graphe : Dijkstra reste préférable dans la majorité des cas, à condition que la renormalisation soit prise en compte.

6 Choix du langage

Java a été choisi comme langage de programmation pour plusieurs raisons. C'est un langage orienté objet, ce qui permet de modéliser naturellement les entités du problème : noeuds, arcs et graphes sous forme de classes. Java offre également de bonnes performances par rapport à Python, tout en restant plus accessible qu'un langage bas niveau comme C ou C++. De plus, la bibliothèque standard de Java fournit des structures de données utiles comme `ArrayList` et `PriorityQueue`, qui ont été directement utilisées dans l'implémentation. Enfin, une certaine familiarité avec le langage a facilité le développement.

7 Résultats

Le programme principale (Main.java) prend en entrée un fichier texte décrivant le graphe. Les exemples utilisés se trouvent dans le dossier **Exemples/** et peuvent être exécutés avec la commande :

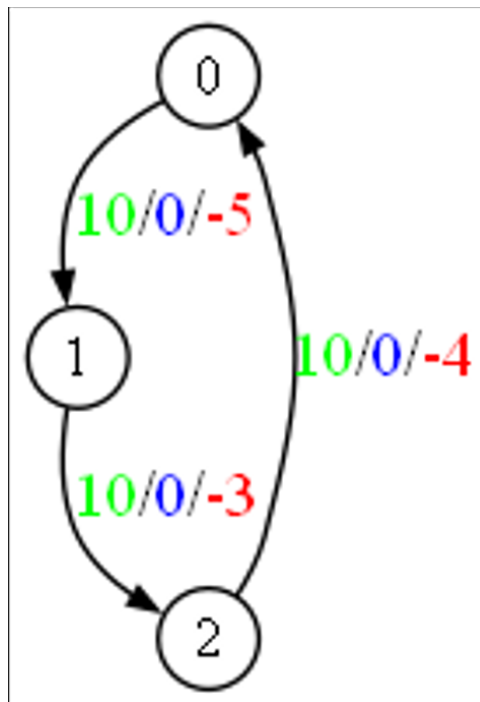
```
java -cp bin Main Exemples/<nom_fichier>
```

Le programme calcule et affiche le flot maximum, la coupe minimale, et le flot à coût minimum avec les deux variantes.

Note : dans les graphes présentés, les chiffres en vert représentent la capacité totale, en bleu la quantité de flot, et en rouge le coût.

7.1 Exemple 1 : Détection de cycle négatif

Le graphe suivant contient un cycle négatif :

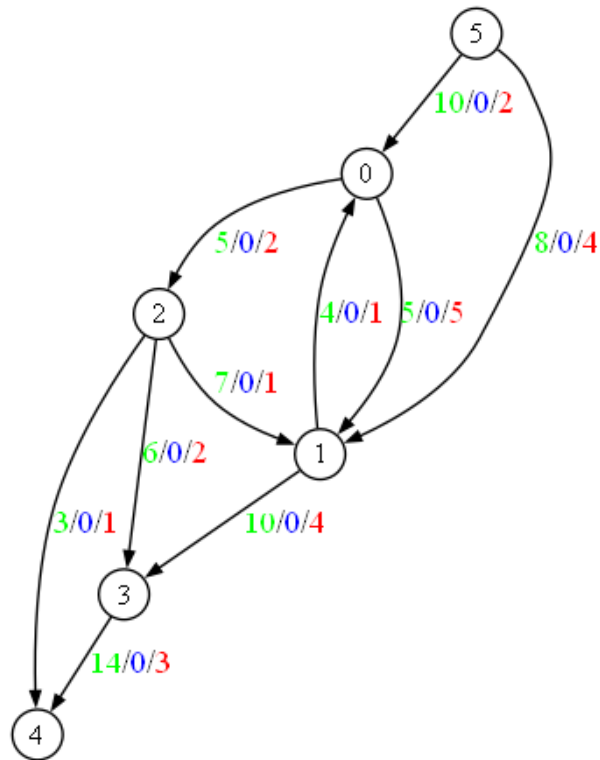


Lors du calcul du flot à coût minimum, un cycle négatif est détecté et l'algorithme s'arrête en levant une exception.

```
Exception in thread "main" java.lang.Exception: Negative cycle detected!  
    at MinCostGraph.min_cost_flow_bellman_ford(MinCostGraph.java:79)  
    at Main.main(Main.java:48)
```

7.2 Exemple 2 : Graphe moyen

Ce graphe est initialement de cette forme :



Les résultats obtenus sont les suivants :

Max Flow (Ford-Fulkerson): 15

Min Cut:

(0 -> 2, max_capacity: 5, flow: 5)

(1 -> 3, max_capacity: 10, flow: 10)

Min Cost Flow (Bellman-Ford): 149

Min Cost Flow (Dijkstra): 149

Par conclusion, voici les resultats obtenus pour Max Flow et Min Cost Flow Algorithmes

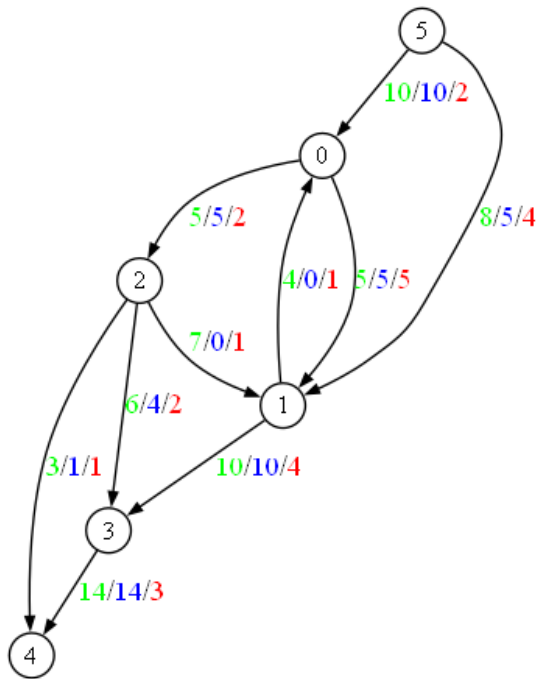


FIGURE 1 – Après Ford-Fulkerson

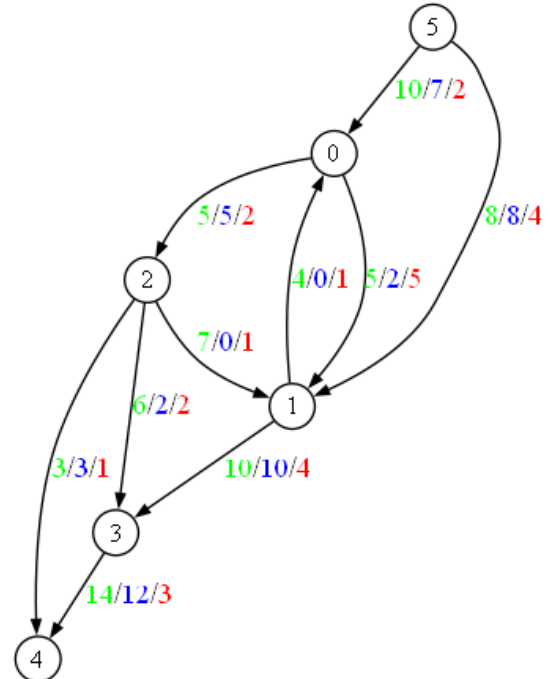


FIGURE 2 – Après Min Cost Flow

7.3 Exemple 3 : Grand graphe

Ce graphe est initialement de cette forme :

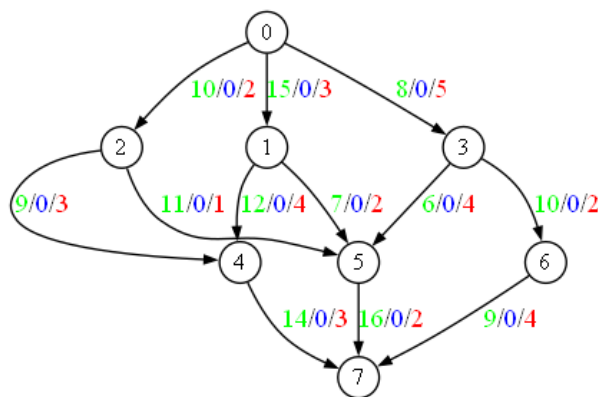


FIGURE 3 – Grand graphe : état initial

Les résultats obtenus sont les suivants :

Max Flow (Ford-Fulkerson): 33

Min Cut:

(0 -> 1, max_capacity: 15, flow: 15)

(0 -> 2, max_capacity: 10, flow: 10)

(0 -> 3, max_capacity: 8, flow: 8)

Min Cost Flow (Bellman-Ford): 270

Min Cost Flow (Dijkstra): 270

Par conclusion, voici les resultats obtenus pour Max Flow et Min Cost Flow Algorithmes :

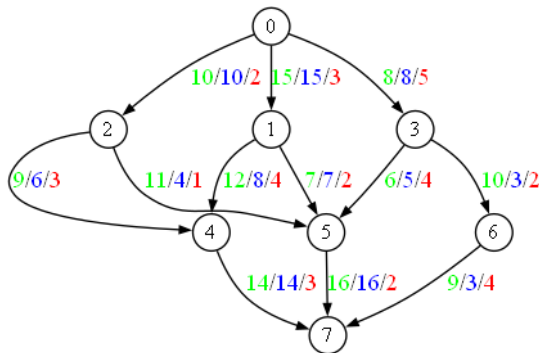


FIGURE 4 – Après Ford-Fulkerson

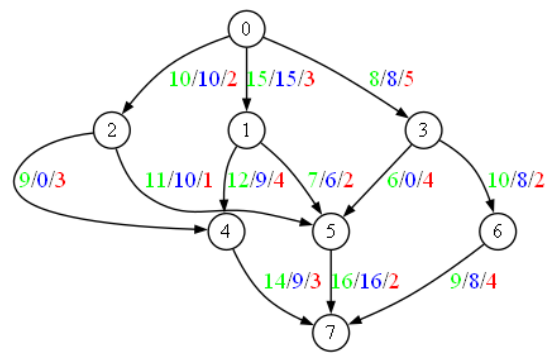


FIGURE 5 – Après Min Cost Flow

8 Conclusion

Dans ce projet, nous avons implémenté les algorithmes de flot maximum, de coupe minimale et de flot à coût minimum. Les deux variantes du flot à coût minimum produisent les mêmes résultats, ce qui confirme la correction de l'implémentation. En termes de performance, Dijkstra est théoriquement plus efficace que Bellman-Ford. Enfin, la détection de cycles négatifs permet d'arrêter l'algorithme avant qu'il ne produise un résultat incorrect.